

Классификация Флинна

Самой ранней и наиболее известной является классификация архитектур вычислительных систем, предложенная в 1966 году Майкл Дж. Флинн (Michael J. Flynn). Классификация базируется на понятии потока, под которым понимается последовательность элементов, команд или данных, обрабатываемая процессором. На основе числа потоков команд и потоков данных Флинн выделяет четыре класса архитектур: SISD, MISD, SIMD, MIMD.

SISD (single instruction stream / single data stream) - одиночный поток команд и одиночный поток данных. К этому классу относятся, прежде всего, классические последовательные машины, или иначе, машины фон-неймановского типа, например, PDP-11 или VAX 11/780. В таких машинах есть только один поток команд, все команды обрабатываются последовательно друг за другом и каждая команда инициирует одну операцию с одним потоком данных. Не имеет значения тот факт, что для увеличения скорости обработки команд и скорости выполнения арифметических операций может применяться конвейерная обработка - как машина CDC 6600 со скалярными функциональными устройствами, так и CDC 7600 с конвейерными попадают в этот класс.

MISD (multiple instruction stream / single data stream) — Чистые, канонические примеры MISD практически не существуют. Это самая редкая и в значительной степени теоретическая категория из классификации Флинна. Её сложно реализовать на практике, так как редко возникает задача, где с одними данными нужно делать совершенно разные операции параллельно.

MIMD (multiple instruction stream / multiple data stream) — множественный поток команд и множественный поток данных. Этот класс предполагает, что в вычислительной системе есть несколько устройств обработки команд, объединенных в единый комплекс и работающих каждое со своим потоком команд и данных.

SIMD

SIMD – это аббревиатура, соответствующая выражению *Single Instruction Stream, Multiple Data* (один поток выполнения инструкций, много потоков данных). Термин SIMD предложил Майкл Дж. Флинн (Michael J. Flynn) для обозначения функциональности, позволяющей выполнять одну инструкцию для нескольких «потоков» данных. Обладая потенциальными возможностями параллельных вычислений, SIMD инструкции могут использоваться для повышения скорости вычислений в широком спектре прикладных областей, таких как обработка изображений, звука, сигналов, для векторных и матричных вычислений и т.д.

SIMD компьютеры состоят из одного командного процессора (управляющего модуля), называемого контроллером, и нескольких модулей обработки данных, называемых процессорными элементами. Управляющий модуль принимает, анализирует и выполняет команды. Если в команде встречаются данные, контроллер рассылает на все процессорные элементы команду, и эта команда выполняется на нескольких или на всех процессорных элементах.

Каждый элемент имеет свою собственную память для хранения данных. Одним из преимуществ данной архитектуры считается то, что в этом случае более эффективно реализована логика вычислений. До половины логических инструкций обычного процессора связано с управлением выполнением машинных команд, а остальная их часть относится к работе с внутренней памятью процессора и выполнению арифметических операций. В SIMD компьютере управление выполняется контроллером, а «арифметика» отдана процессорным элементам.

MMX

Первой реализацией SIMD был набор инструкций MMX или Multi Media Extension (мультимедийное расширение). MMX впервые появился в процессорах Pentium MMX в 1997 году. Для расширения были внедрены дополнительные регистры размером 64 бита и с названиями mm0 — mm7. Одна инструкция может применяться к двум 32-битным целым числам, четырем 16-битным целым числам или восьми 8-битным целым числам одновременно. MMX поддерживал только целочисленные вычисления.

MMX также присутствовал в процессорах AMD и Cyrix в расширенном виде MMX Extended. Чуть позже AMD представил улучшенную модификацию MMX под брендовым названием 3DNow!. 3DNow! позволял ускорить производительность многих приложений с интенсивной графикой за счет реализации работы с числами с плавающей точкой с использованием векторных регистров. Технология 3DNow! расширяла возможности технологии MMX, не требуя введения новых режимов работы процессора и новых регистров. Первым микропроцессором, реализовавшим 3DNow!, стал AMD K6-2, который был представлен в 1998 году.

SSE

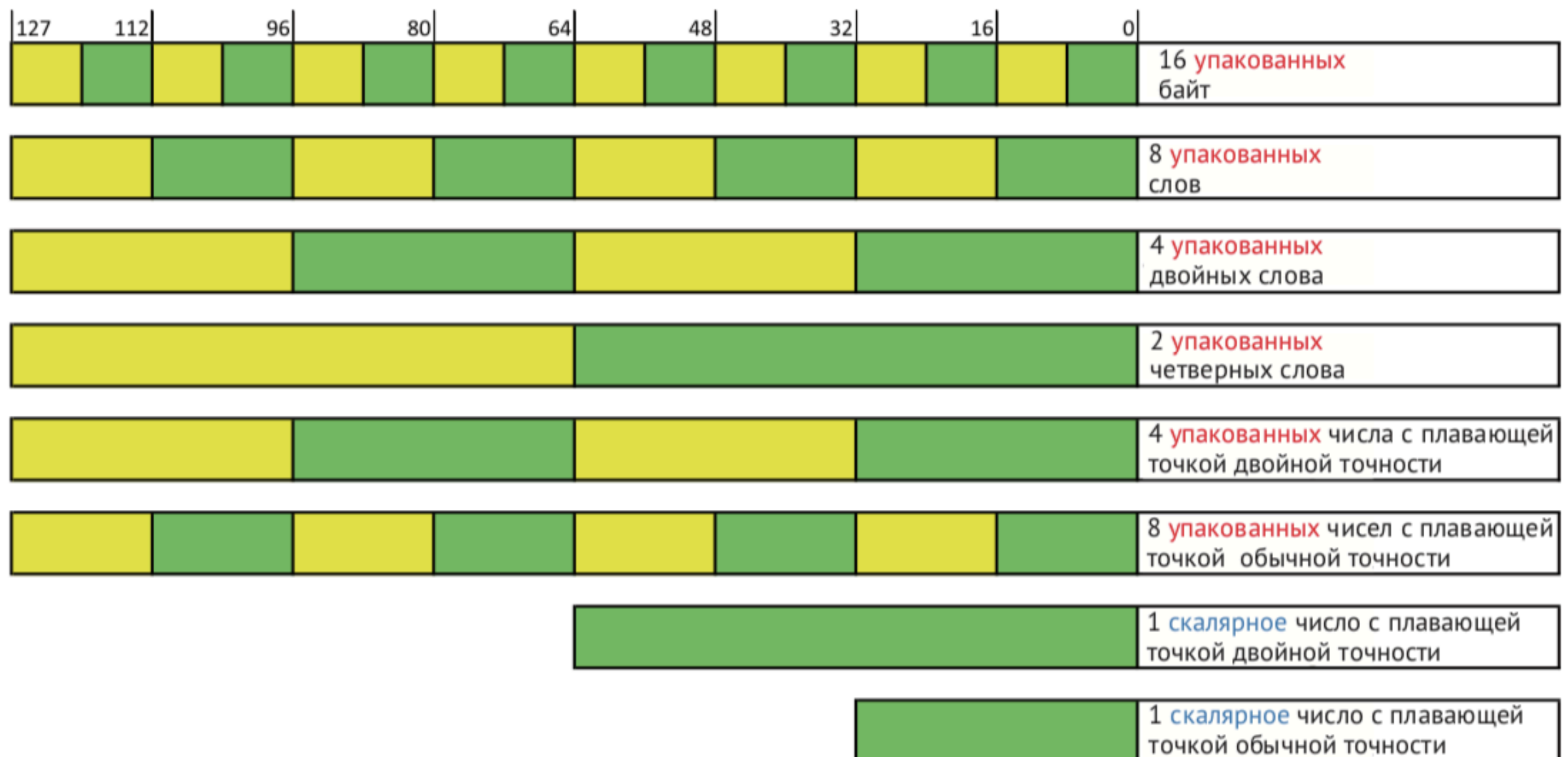
Набор инструкций MMX был заменен набором инструкций Streaming SIMD Extension (SSE). Streaming SIMD Extension (SSE) — набор инструкций, разработанный Intel и впервые представленный в процессорах серии Pentium III как ответ на аналогичный набор инструкций 3DNow! от AMD, который был представлен годом раньше. Процессор, поддерживающий функциональность SSE, имеет 16 дополнительных 128-битовых регистров (от xmm0 до xmm15) и управляющий регистр mxcsr.

Регистры `xmm` могут содержать скалярные данные (scalar data) или упакованные данные (packed data). Под скалярными данными подразумевается только одно значение, занимающее целиком регистр. Если число помещается в регистр `xmm0`, то `xmm0` содержит скалярное значение. Но в `xmm0` также можно записать несколько значений, тогда эти значения определяются как упакованные данные.

Возможные варианты сохранения значений в любом xmm-регистре:

- два 64-битовых числа с плавающей точкой двойной точности;
- четыре 32-битовых числа с плавающей точкой обычной (одинарной) точности;
- два 64-битовых целых числа (четверные слова);
- четыре 32-битовых целых числа (двойные слова);
- восемь 16-битовых коротких (short) целых чисел (слова);
- шестнадцать 8-битовых байт или символов.

Для скалярных значений и для упакованных значений существуют различные ассемблерные инструкции. В руководствах Intel можно найти большое количество описаний доступных SSE-инструкций.



Выровненные данные

Данные в памяти могут быть выровненными по определённым адресам, кратным 16, 32 и т. д.

Выравнивание данных в памяти может существенно улучшить производительность программы. Причина в следующем: для обработки выровненных упакованных данных SSE-инструкции требуют извлечения 16-байтовых фрагментов памяти за одну операцию. Если данные в памяти не выровнены, то процессору потребуется более одной операции для извлечения, таким образом, выполнение замедляется.

Существуют два типа SSE-инструкций: инструкции для выровненных упакованных данных и инструкции для невыровненных упакованных данных. Инструкции для не выровненных упакованных данных могут работать с невыровненными фрагментами памяти, но в общем случае это ухудшает производительность.

Для типов данных, имеющих более жёсткие ограничения по выравниванию, например, данные SSE (16 байт), либо в случае когда требуется выравнивать блоки по строкам кэша или страницам памяти, нет готового стандартного способа получить корректно выровненную память. Начиная со стандарта C11 появилась функция `void *aligned_alloc( size_t alignment, size_t size )`, которая позволяет явно указать желаемое выравнивание (число, которому должен быть кратен адрес блока). Возвращённый этой функцией указатель можно передать в `free` (для освобождения) или в `realloc` (однако для изменения размера блока, `realloc` не гарантирует сохранения выравнивания!), так же, как если бы блок был выделен вызовом `malloc`.

Если не ориентироваться на C11, то можно рассмотреть варианты, предлагаемые конкретными платформами и на неоптимальный вариант на основе стандартных функций управления динамической памятью.

В случае платформы POSIX (`_POSIX_VERSION` $\geq$ 200112L) существует функция `posix_memalign`, результат которой можно освободить с помощью стандартной функции `free`.

```
// POSIX version
```

```
#include <stdlib.h>
```

```
// возвращает код ошибки или 0, в случае успеха
```

```
// адрес выделенного блока записывается по указателю memptr
```

```
int posix_memalign(void **memptr, size_t  
alignment, size_t size);
```

Свои функции предлагает Microsoft Visual C++ runtime.
Это `_aligned_malloc`, `_aligned_free` и
`_aligned_realloc`.

```
void * _aligned_malloc( size_t size, size_t  
alignment );
```

```
void _aligned_free ( void *memblock );
```

```
void * _aligned_realloc( void *memblock, size_t  
size, size_t alignment );
```

Подробности по ссылке: <https://docs.microsoft.com/ru-ru/cpp/c-runtime-library/reference/aligned-malloc?view=msvc-170>

Документация к компилятору Intel предлагает аналогичные функции с другими названиями.

```
// ICC version
#include <malloc.h>

void* _mm_malloc(size_t size, size_t
alignment);

void _mm_free(void*)
```

Подробности по ссылке: <https://software.intel.com/sites/products/documentation/doclib/iss/2013/compiler/cpp-lin/GUID-D0927A8E-A220-4F50-8697-C89BBE6EFC95.htm>

Если варианты, приведённые выше, не подходят, по каким либо причинам, то можно реализовать выравнивание самостоятельно. Для этого требуется выделить блок размера = запрошенный размер + размер указателя + выравнивание — $\min(\text{гарантированное выравнивание, размер указателя})$, выбрать в нём наименьший выровненный адрес (адрес результата) и записать адрес блока в указатель прямо перед адресом результата, чтобы затем извлечь его при удалении блока. Недостаток данного метода заключается в перерасходе памяти, который может превышать 100% в неудачных случаях (т.е. более чем двойной расход). Кроме того, может замусориваться кэш (обращение к предыдущей строке или странице для сохранения указателя на реальный блок).

```
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>

void* custom_aligned_alloc(size_t size, size_t alignment){
    void *mem = malloc(size + alignment + sizeof(void*));
    void **ptr = (void**)((uintptr_t)( mem + alignment + sizeof(void*) ) &
~( alignment - 1 ));
    *(ptr - 1) = mem;
    printf("Malloc %p, Offset %ld\n", mem, (unsigned long)((void*)ptr) -
(unsigned long)mem);
    return ptr;
}

void aligned_free(void *ptr) {
    printf("Free %p \n", *(((void**)ptr) - 1));
    free(((void**)ptr)[-1]);
}

int main(){
    void *data = custom_aligned_alloc(4096, 4096);
    aligned_free(data);
}
```

Выравнивание также возможно с помощью специальных указаний компилятору, не входящих в состав стандарта языка. Например главной особенностью GNU C является механизм `__attribute__`. С помощью `__attribute__` возможно установить атрибут функции (`Function Attribute`), атрибут переменной (`Variable Attribute`) и атрибут типа (`Type Attribute`). Синтаксическая запись `__attribute__` использует два подчеркивания до и после `attribute`. Далее следует пара скобок, внутри которых находятся соответствующие параметры.

`__attribute__ ( ( список атрибутов ) )`

Пример атрибута, устанавливающего выравнивание структуры по 8 байт:

```
struct example_s{  
    short b[3];  
} __attribute__((aligned (4)));
```

Если размер `short` равен 2 байта, то размер `example_s` равен 6. Благодаря атрибуту компилятор установит выравнивание `example_s` в 4 байта, а размер структуры будет увеличен до 8 байт.

Использование атрибута упаковки данных может уменьшить пространство, занимаемое объектом. Пример упаковки структуры с помощью атрибута `__packed__`.

```
struct unpacked_struct
{
    char c;
    int i;
};
```

```
struct packed_struct
{
    char c;
    int i;
    struct unpacked_struct s;
}__attribute__((__packed__));
```


Если используются запакованные структуры, то обращаться к полям объекта структуры, созданной на стеке можно с помощью оператора точки, это не вызовет ошибки. Однако, необходимо понимать, что доступ к данным, располагающимся с «не правильным» смещением может быть дольше, чем обычно.

Атрибуты позволяют задать выравнивание для статических массивов и массивов, созданных на стеке.

Например буфер из 256 символов `char`, выровненных по 4096, то есть фактически находящихся внутри одной страницы памяти:

```
char buffer[256] __attribute__((aligned(4096)));
```

Microsoft Visual C++ runtime использует похожий синтаксис для атрибутов. Для Microsoft Visual C++ runtime `__declspec` это ключевое слово, указывающее, что экземпляр заданного типа должен храниться с атрибутом класса хранения, указанным ниже:

```
__declspec(align(4096)) char buffer[256];
```

Методы применения SIMD на практике

Существуют несколько способов использования SIMD инструкций:

1. Самый простой способ — использовать кросс-платформенную библиотеку функций (IPP, Intel Integrated Performance Primitives), разработанную Intel. Реализации функций внутри оптимизируются с помощью инструкций SIMD.
2. Автоматическая векторизация компилятора.
3. Директивы компилятора, такие как `#pragma simd` и `#pragma omp simd` в OpenMP.
4. Использовать встроенные функции — интринсики (`intrinsics`).
5. Вставки на ассемблере.
6. Использование ассемблера для непосредственного управления регистрами.

Поддерживаемые SIMD операции

1. Чтение, запись
2. Арифметические, логические
3. Перестановки, копирование элементов вектора
4. Сравнение
5. Преобразования типов
6. Специальные операции

Пример SSE

```
extern printf
section .data
    spvector1 dd 1.1
               dd 2.2
               dd 3.3
               dd 4.4
    spvector2 dd 5.1
               dd 6.2
               dd 7.3
               dd 8.4
    fmt1 db "Sum of not align Single Precision Vector 1 and
Vector 2:"
          db " %f, %f, %f, %f",10,0
section .bss
    spvector_res resd 4

section .text
    global main
```

main:

push rbp

mov rbp, rsp

; Сложение 2 векторов из чисел с плав. точкой с обычной точностью.

movups xmm0, [spvector1]

movups xmm1, [spvector2]

addps xmm0, xmm1

movups [spvector_res], xmm0

mov rsi, spvector_res

mov rdi, fmt1

call printspfp

leave

ret

printspfp:

push rbp

mov rbp, rsp

movss xmm0, [rsi]

cvtss2sd xmm0, xmm0

movss xmm1, [rsi+4]

cvtss2sd xmm1, xmm1

movss xmm2, [rsi+8]

cvtss2sd xmm2, xmm2

movss xmm3, [rsi+12]

cvtss2sd xmm3, xmm3

mov rax, 4

call printf

leave

ret

Данные в разделах `section .data` и `section .bss` должны быть выровнены по 16-байтовой границе. Можно воспользоваться директивами ассемблера `align 16` и `alignb 16` перед данными (NASM), которые необходимо выровнять.

Программа написана на ассемблере NASM 64. Программа состоит из нескольких секций. Первая секция это `data`, которая необходима для объявления инициализированных данных или констант. Данные в этой секции не могут быть изменены во время выполнения программы. Секция `bss` используется для объявленных переменных. Секция `text` используется для хранения кода программы. Данная секция должна начинаться с объявления `global main`, сообщающая откуда нужно начинать выполнение программы. Для секции `data` возможно выделение данных с помощью квалификаторов `DB` — 1 байт, `DW` — 2 байта, `DD` — 4 байта, `DQ` — 8 байт, а для секции `bss` данные выделяются с помощью `RESB` — 1 байт, `RESW` — 2 байта, `RESD` — 4 байта и `RESQ` — 8 байт.

Первая SSE инструкция `movups` (означающая «`move unaligned packed single precision`» – перемещение невыровненных упакованных данных с обычной точностью) копирует данные из памяти в регистры `xmm0` и `xmm1`. В результате регистр `xmm0` содержит один вектор с четырьмя значениями с обычной точностью, а регистр `xmm1` содержит второй вектор с четырьмя значениями с обычной точностью. Потом применяется инструкция `addps` (означающая «`add packed single precision`» – сложение упакованных данных с обычной точностью) для сложения этих векторов, а полученный в результате вектор записывается в регистр `xmm0`, затем передается в память. Далее результат выводится с помощью функции `printsfpr`. В этой функции копируется каждое значение из памяти в `xmm`-регистры с помощью инструкции `movss` (означающей «`move scalar single precision`» – перемещение скалярного значения с обычной точностью). Значения выводятся на экран с помощью `си`-библиотечной функции `printf`, которая линкуется в момент компиляции примера.

Поскольку функция `printf` ожидает передачу аргументов с плавающей точкой двойной точности, выполняется преобразование чисел с плавающей точкой обычной точности в числа двойной точности с помощью инструкции `cvtss2sd` (означающей «convert scalar single to scalar double» – преобразование скалярного значения обычной точности в скалярное значение двойной точности).

Далее код, складывающий выровненный вектор из четырёх чисел типа `float`.

```
extern printf
section .data
    dummy db 13 ; случайное смещение памяти
align 16
    spvector1 dd 1.1
                dd 2.2
                dd 3.3
                dd 4.4
    spvector2 dd 5.1
                dd 6.2
                dd 7.3
                dd 8.4
    fmt1 db "Sum of align Single Precision Vector 1 and Vector 2:"
          db " %f, %f, %f, %f",10,0
section .bss
alignb 16
    spvector_res resd 4

section .text
    global main
```

main:

```
push rbp
mov rbp, rsp
```

; Сложение 2 векторов из чисел с плав. точкой с обычной точностью.

```
movaps    xmm0, [spvector1]
addps     xmm0, [spvector2]
movaps     [spvector_res], xmm0
mov        rsi, spvector_res
mov        rdi, fmt1
call       printspfp
leave
ret
```

printspfp:

```
push rbp
mov rbp, rsp
movss     xmm0, [rsi]
cvtss2sd  xmm0, xmm0
movss     xmm1, [rsi+4]
cvtss2sd  xmm1, xmm1
movss     xmm2, [rsi+8]
cvtss2sd  xmm2, xmm2
movss     xmm3, [rsi+12]
cvtss2sd  xmm3, xmm3
mov        rax, 4
call       printf
leave
ret
```

В начале этой программы создается переменная `dummy` для полной уверенности в том, что память не выровнена по 16 байтовой границе. Затем используется директива `align 16` в разделе `section .data` и директива `alignb 16` в разделе `section .bss`. Необходимо добавлять эти директивы ассемблера перед каждым блоком данных, который требует выравнивания. SSE-инструкции немного отличаются от инструкций из версии без выравнивания. Инструкция `movaps` (означающая «`move aligned packed single precision`» – перемещение выровненных упакованных данных обычной точности) копирует данные из памяти в регистр `xmm0`. После этого сразу можно выполнять сложение упакованных чисел в памяти с упакованными значениями в регистре `xmm0`. В этом состоит отличие от версии без выравнивания, где требовалась предварительная запись двух значений в `xmm`-регистры.

Если добавить переменную `dummy` в версию без выравнивания и попытаться использовать инструкцию `movaps` вместо `movups` с переменной памяти как вторым операндом, то возникает опасность нарушения сегментации памяти (`segmentation fault`) во время выполнения. Регистр `xmm0` содержит полученный в результате сложения вектор с четырьмя значениями обычной точности. Затем выводится этот результат с помощью функции `printfp`. В этой функции каждое значение из памяти копируется в `xmm`-регистры. Поскольку функция `printf` ожидает передачу аргументов с плавающей точкой двойной точности, выполняется преобразование чисел с плавающей точкой обычной точности в числа двойной точности с помощью инструкции `cvtss2sd`.

AVX

Advanced Vector Extensions (AVX) – это расширение системы команд для процессоров x86. Система SSE предлагает 16 xmm-регистров размером 128 бит каждый, тогда как система AVX предоставляет 16 ymm-регистров размером 256 бит каждый. Нижняя (младшая) половина каждого ymm-регистра в действительности представляет собой соответствующий xmm-регистр. AVX 512 – это дальнейшее расширение, предоставляющее 32 zmm-регистра размером 512 бит каждый. В дополнение к ymm-регистрам система AVX расширяет инструкции SSE и предоставляет набор дополнительных новых инструкций.

Примеры AVX

Пример кода на ассемблере, складывающий невыровненный вектор из восьми чисел типа float.

```
extern printf
```

```
section .data
```

```
    spvector1 dd 1.1
```

```
            dd 2.1
```

```
            dd 3.1
```

```
            dd 4.1
```

```
            dd 5.1
```

```
            dd 6.1
```

```
            dd 7.1
```

```
            dd 8.1
```

```
    spvector2 dd 1.1
```

```
            dd 2.1
```

```
            dd 3.1
```

```
            dd 4.1
```

```
            dd 5.1
```

```
            dd 6.1
```

```
            dd 7.1
```

```
            dd 8.1
```

```
    fmt1      db "Sum of not align Single Precision Vector 1 and Vector 2 with AVX:"
```

```
            db " %f, %f, %f, %f, %f, %f, %f, %f",10,0
```

```
section .bss
```

```
    spvector_res resd 8
```



```
section .text
    global main
main:
    push rbp
    mov rbp, rsp
    vmovups    ymm0, [spvector1]
    vmovups    ymm1, [spvector2]
    vaddps     ymm2, ymm0, ymm1
    vmovups    [spvector_res], ymm2
    mov        rsi, spvector_res
    mov        rdi, fmt1
    call       printspfp
    leave
    ret
```

```
printspfp:
    push rbp
    mov rbp, rsp
    movss      xmm0, [rsi]
    cvtss2sd    xmm0, xmm0
    movss      xmm1, [rsi+4]
    cvtss2sd    xmm1, xmm1
    movss      xmm2, [rsi+8]
    cvtss2sd    xmm2, xmm2
    movss      xmm3, [rsi+12]
    cvtss2sd    xmm3, xmm3
    movss      xmm4, [rsi+16]
    cvtss2sd    xmm4, xmm4
    movss      xmm5, [rsi+20]
    cvtss2sd    xmm5, xmm5
    movss      xmm6, [rsi+24]
    cvtss2sd    xmm6, xmm6
    movss      xmm7, [rsi+28]
    cvtss2sd    xmm7, xmm7
    mov        rax, 4
    call       printf
    leave
```

Для суммирования вектора используются новые инструкции с тремя операндами, как показано ниже:

```
vaddps ymm2,ymm0,ymm1
```

То есть при сложении данные из первого операнда не «портятся», а результат записывается в `ymm2`. Функции вывода просто загружают значения из памяти в `xmm` регистр, выполняют преобразование чисел с обычной точностью в числа с двойной точностью, когда это необходимо, затем вызывают функцию `printf`.

Пример кода на ассемблере, складывающий выровненный вектор из восьми чисел типа float.

```
extern printf
section .data
    dummy db 13 ; случайное смещение памяти
align 16
    spvector1 dd 1.0
               dd 2.1
               dd 3.1
               dd 4.1
               dd 5.1
               dd 6.1
               dd 7.1
               dd 8.1

    spvector2 dd 1.1
               dd 2.1
               dd 3.1
               dd 4.1
               dd 5.1
               dd 6.1
               dd 7.1
               dd 8.1

    fmt1      db "Sum of not align Single Precision Vector 1 and Vector 2 with AVX:"
               db " %f, %f, %f, %f, %f, %f, %f, %f",10,0
section .bss
alignb 16
    spvector_res resd 8
```

```
section .text
    global main
main:
    push rbp
    mov rbp, rsp
    vmovups    ymm0, [spvector1]
    vaddps     ymm2, ymm0, [spvector2]
    vmovups    [spvector_res], ymm2

    mov        rsi, spvector_res
    mov        rdi, fmt1
    call       printspfp
    leave
    ret
```

```
printspfp:
    push rbp
    mov rbp, rsp
    movss      xmm0, [rsi]
    cvtss2sd   xmm0, xmm0
    movss      xmm1, [rsi+4]
    cvtss2sd   xmm1, xmm1
    movss      xmm2, [rsi+8]
    cvtss2sd   xmm2, xmm2
    movss      xmm3, [rsi+12]
    cvtss2sd   xmm3, xmm3
    movss      xmm4, [rsi+16]
    cvtss2sd   xmm4, xmm4
    movss      xmm5, [rsi+20]
    cvtss2sd   xmm5, xmm5
    movss      xmm6, [rsi+24]
    cvtss2sd   xmm6, xmm6
    movss      xmm7, [rsi+28]
    cvtss2sd   xmm7, xmm7
    mov        rax, 4
    call       printf
    leave
    ret
```

В начале программы опять используется переменная, для случайного смещения памяти. Далее следует выравнивание памяти. Выровненная версия кода позволяет избавиться от одной из операции копирования в `ymm`-регистр, а забрать данные напрямую из памяти.

Некоторые векторные инструкции

Скалярные инструкции: 1 × double

- addsd a + b
- mulsd a * b
- divsd a / b

Векторные SSE2: 2 × double

- addpd a + b
- mulpd a * b
- divpd a / b

Векторные AVX: 4 × double

- vaddpd a + b
- vmulpd a * b
- vdivpd a / b

Векторные AVX2: 4 × double

- vfmadd132pd a*b+c

Скалярные инструкции: 1 × float

- addss a + b
- mulss a * b
- divss a / b

Векторные SSE2: 4 × float

- addps a + b
- mulps a * b
- divps a / b

Векторные AVX: 8 × float

- vaddps a + b
- vmulps a * b
- vdivps a / b

Векторные AVX2: 8 × float

- vfmadd132ps a*b+c

Поддержка SIMD

Выяснить, поддерживает ли процессор то или иное SIMD поколение инструкций можно с помощью функций `__builtin_cpu_supports`, например:

```
printf( "sse: %s\n", __builtin_cpu_supports( "sse" ) ?  
"Yes" : "No" );  
printf( "sse2: %s\n",  
__builtin_cpu_supports( "sse2" ) ? "Yes" : "No" );  
printf( "avx: %s\n", __builtin_cpu_supports( "avx" ) ?  
"Yes" : "No" );  
printf( "avx2: %s\n", __builtin_cpu_supports( "avx2" ) ?  
"Yes" : "No" );  
printf( "avx512f: %s\n",  
__builtin_cpu_supports( "avx512f" ) ? "Yes" : "No" );
```

Также узнать информацию о поддерживаемых расширениях позволяет файл `/proc/cpuinfo`.

Интринсики SIMD

Можно получить доступ к векторным инструкциям и в программе на языке высокого уровня (в частности, C/C++) без ассемблерных вставок. Для этого используются так называемые интринсики (`intrinsics`) — встроенные объекты компилятора. В заголовочном файле объявлен один или несколько типов данных (с точки зрения программиста, это массив фиксированной длины, но без возможности доступа к элементам этого массива), переменной одного из этих типов соответствует векторный регистр. Также в заголовочном объявлены функции, которые принимают аргументы, возвращают значения указанных выше типов и производят с точки зрения программиста те же операции над данными, что и соответствующие им векторные инструкции. В действительности же настоящей программной реализации этих функций не существует: компилятор при генерации объектного кода заменяет вызов функции векторной инструкцией. Таким образом, с помощью интринсиков можно написать программу на языке высокого уровня, близкую или равную по производительности программе, написанной на языке ассемблера.

Для доступа к интринсикам достаточно подключить соответствующий заголовочный файл. Интринсики не являются частью стандартов языков C/C++, но поддерживаются основными компиляторами: GCC, Clang, MSVC, Intel. Интринсики позволяют также упорядочить работу с разнообразными типами обрабатываемых данных. Важно отметить, что процессору (по крайней мере, в архитектуре x86) недоступна информация о типе данных, содержащихся в регистре. При выполнении той или иной векторной инструкции они интерпретируются как данные определённого, связанного с инструкцией типа: числа с плавающей запятой, целого числа некоторой разрядности со знаком или без знака. Контролировать корректность производимых вычислений при этом должен программист, что требует немалого внимания. Тем более, иногда тип данных может изменяться: например, при целочисленном умножении разрядность произведения равна сумме разрядностей множителей. Интринсики позволяют в некоторой степени упростить задачу.

Тип данных обычно называется `__mxxx (T)`, где `xxx` представляет количество битов данных, например `__m128`, предоставляемый SSE, равен 128 битам, а `__m256`, предоставляемый AVX, является 256 бит. `T` - это тип, если это тип с плавающей запятой одинарной точности, он опускается, если это целое число, это `i`, например `__m128i`, если это тип с плавающей точкой двойной точности, это `d`, например `__m256d`.

Так, векторным регистрам `xmm` (SSE) соответствуют три типа данных:

___`m128`, «массив» из четырёх чисел с плавающей запятой с одинарной точностью;

___`m128d`, «массив» из двух чисел с плавающей запятой двойной точности;

___`m128i`, 128-битный регистр, который можно рассматривать как «массив» 8-, 16-, 32- и 64-битных чисел. Поскольку конкретная векторная инструкция работает, как правило, с одним из трёх типов данных (число с плавающей запятой одинарной точности, двойной точности, целочисленный), то и аргументы функций-векторных инструкций имеют один из трёх указанных типов.

Подобным образом устроена и система типов AVX2: в ней имеются типы `__m256` (с плавающей запятой), `__m256d` (двойной точности) и `__m256i` (целочисленный). Соответственно типы `__m256` размещаются в 256 байтном `ymm`. Нижняя (младшая) половина каждого `ymm`-регистра в действительности представляет собой соответствующий `xmm`-регистр.

Интринсики NEON реализуют ещё более развитую систему типов. 128-битному регистру соответствуют типы `int32x4_t`, `int16x8_t`, `int8x16_t`, `float32x4_t`, `float64x2_t`. В NEON имеются и типы данных, содержащие несколько регистров, например `int8x16x2_t`. В подобной системе в любой момент известны конкретный тип и разрядность содержимого регистра, и меньше вероятность допустить ошибку там, где преобразуется тип данных и меняется их разрядность.

Инструкции SSE / AVX в основном определены в следующих файлах заголовков:

`<xmmintrin.h>`: SSE, поддерживает одновременные операции с 4 32-битными числами с плавающей запятой одинарной точности.

`<emmintrin.h>`: SSE 2, поддерживает одновременные операции с двумя 64-битными числами с плавающей запятой двойной точности.

`<pmmintrin.h>`: SSE 3, поддерживает горизонтальные операции с регистрами SIMD, такими как `hadd`, `hsub` и т. д.

`<tmmintrin.h>`: SSSE 3, добавлены дополнительные инструкции.

`<smmintrin.h>`: SSE 4.1, поддерживает умножение точек и другие операции формирования.

`<nmmintrin.h>`: SSE 4.2, добавлены дополнительные инструкции. (Это поддерживает все предыдущие версии SSE)

`<immintrin.h>`: AVX, поддерживает одновременную работу с 8 числами с плавающей запятой одинарной точности или 4 числами с плавающей запятой двойной точности.

Для GCC существует заголовочный файл `<x86intrin.h>`, который включает сразу все интринсики.

Встроенные функции представляют собой следующий паттерн: `_mm (xxx) _name_PT`, где `xxx` - это количество бит регистра `SIMD`, если 128 бит, то `xxx` опускается, например `_mm_addsub_ps`, если 256 бит, то 256, например `_mm256_add_ps`, `name` - это имя операции, выполняемой функцией, например `_mm_add_ps` для сложения и `_mm_sub_ps` для вычитания. Встроенных функций достаточно много, однако на сайте Intel есть удобное веб-приложение, которое позволяет найти требуемые для тех или иных задач функции.

Ссылка: <https://www.intel.com/content/www/us/en/docs/intrinsics-guide/index.html?wapkw=intrinsics%20guide>

Ссылка: <https://www.laruence.com/sse/>

Сайт позволяет выделить те расширения, с которыми связаны требуемые функции с помощью галочек или начать писать имя в поисковой строке. Каждой функции соответствует описание, аргументы, возвращаемое значение.



?

Technologies

- ☐ MMX
- ☐ SSE
- ☐ SSE2
- ☐ SSE3
- ☐ SSSE3
- ☐ SSE4.1
- ☐ SSE4.2
- ☐ AVX
- ☐ AVX2
- ☐ FMA
- ☐ AVX-512
- ☐ KNC
- ☐ SVML
- ☐ Other

Categories

- ☐ Application-Targeted
- ☐ Arithmetic
- ☐ Bit Manipulation
- ☐ Cast
- ☐ Compare
- ☐ Convert
- ☐ Cryptography
- ☐ Elementary Math Functions

<code>void _mm_2intersect_epi32 (__m128i a, __m128i b, __mmask8* k1, __mmask8* k2)</code>	<code>vp2intersectd</code>
<code>void _mm256_2intersect_epi32 (__m256i a, __m256i b, __mmask8* k1, __mmask8* k2)</code>	<code>vp2intersectd</code>
<code>void _mm512_2intersect_epi32 (__m512i a, __m512i b, __mmask16* k1, __mmask16* k2)</code>	<code>vp2intersectd</code>
<code>void _mm_2intersect_epi64 (__m128i a, __m128i b, __mmask8* k1, __mmask8* k2)</code>	<code>vp2intersectq</code>
<code>void _mm256_2intersect_epi64 (__m256i a, __m256i b, __mmask8* k1, __mmask8* k2)</code>	<code>vp2intersectq</code>
<code>void _mm512_2intersect_epi64 (__m512i a, __m512i b, __mmask8* k1, __mmask8* k2)</code>	<code>vp2intersectq</code>
<code>__m512i _mm512_4dpwssd_epi32 (__m512i src, __m512i a0, __m512i a1, __m512i a2, __m512i a3, __m128i * b)</code>	<code>vp4dpwssd</code>
<code>__m512i _mm512_mask_4dpwssd_epi32 (__m512i src, __mmask16 k, __m512i a0, __m512i a1, __m512i a2, __m512i a3, __m128i * b)</code>	<code>vp4dpwssd</code>
<code>__m512i _mm512_maskz_4dpwssd_epi32 (__mmask16 k, __m512i src, __m512i a0, __m512i a1, __m512i a2, __m512i a3, __m128i * b)</code>	<code>vp4dpwssd</code>
<code>__m512i _mm512_4dpwssds_epi32 (__m512i src, __m512i a0, __m512i a1, __m512i a2, __m512i a3, __m128i * b)</code>	<code>vp4dpwssds</code>
<code>__m512i _mm512_mask_4dpwssds_epi32 (__m512i src, __mmask16 k, __m512i a0, __m512i a1, __m512i a2, __m512i a3, __m128i * b)</code>	<code>vp4dpwssds</code>
<code>__m512i _mm512_maskz_4dpwssds_epi32 (__mmask16 k, __m512i src, __m512i a0, __m512i a1, __m512i a2, __m512i a3, __m128i * b)</code>	<code>vp4dpwssds</code>
<code>__m512 _mm512_4fmadd_ps (__m512 src, __m512 a0, __m512 a1, __m512 a2, __m512 a3, __m128 * b)</code>	<code>v4fmaddps</code>
<code>__m512 _mm512_mask_4fmadd_ps (__m512 src, __mmask16 k, __m512 a0, __m512 a1, __m512 a2, __m512 a3, __m128 * b)</code>	<code>v4fmaddps</code>
<code>__m512 _mm512_maskz_4fmadd_ps (__mmask16 k, __m512 src, __m512 a0, __m512 a1, __m512 a2, __m512 a3, __m128 * b)</code>	<code>v4fmaddps</code>
<code>__m128 _mm_4fmadd_ss (__m128 src, __m128 a0, __m128 a1, __m128 a2, __m128 a3, __m128 * b)</code>	<code>v4fmaddss</code>

Встроенные функции позволяют писать код на языке высокого уровня, например на Си. Например Простой цикл, в котором складывают два массива 64-битных действительных чисел, для интринсиков будет выглядеть так:

```
double *a = (double*)malloc( SIZE * sizeof(double) );  
double *b = (double*)malloc( SIZE * sizeof(double) );  
double *c = (double*)malloc( SIZE * sizeof(double) );
```

```
double timeStart, timeEnd;  
timeStart = omp_get_wtime();
```

```
for (int i = 0; i < SIZE; i += 4) {  
    __m256d x = _mm256_loadu_pd(&a[i]);  
    __m256d y = _mm256_loadu_pd(&b[i]);  
    __m256d z = _mm256_add_pd(x, y);  
    _mm256_storeu_pd(&c[i], z);  
}
```

```
timeEnd = omp_get_wtime();  
printf( "Время выполнения %lf\n", timeEnd - timeStart);
```

Пример работает с не выровненной памятью. Операции чтения и записи имеют по две версии — `load / loadu` и `store / storeu`. Буква «u» здесь означает «unaligned» или в переводе «невывровненный». Первые корректно работают только тогда, когда весь считываемый блок помещается на одну кэш-линию, в то время как `unaligned` версия работает во всех случаях. Невывровненные версии встроенных функций обычно медленней своих выровненных аналогов, хотя бы потому, что нужно загружать две кэш-линии вместо одной.

Пример с выровненной памятью может выглядеть следующим образом:

```
double *a = (double*)aligned_alloc( 32, SIZE *
sizeof(double) );
double *b = (double*)aligned_alloc( 32, SIZE *
sizeof(double) );
double *c = (double*)aligned_alloc( 32, SIZE *
sizeof(double) );
```

```
double timeStart, timeEnd;
timeStart = omp_get_wtime();
```

```
for (int i = 0; i < SIZE; i += 4) {
    __m256d x = _mm256_load_pd(&a[i]);
    __m256d y = _mm256_load_pd(&b[i]);
    __m256d z = _mm256_add_pd(x, y);
    _mm256_store_pd(&c[i], z);
}
```

```
timeEnd = omp_get_wtime();
printf( "Время выполнения %lf\n", timeEnd - timeStart);
```

Пример сложения массивов целых чисел с невыравненной памятью:

```
for (int i = 3; i + 7 < n; i += 8) {  
    __m256i x = _mm256_loadu_si256((__m256i*) &a[i]);  
    __m256i y = _mm256_loadu_si256((__m256i*) &b[i]);  
    __m256i z = _mm256_add_epi32(x, y);  
    _mm256_storeu_si256((__m256i*) &c[i], z);  
}
```

Пример сложения массивов целых чисел с выравненной памятью:

```
for (int i = 0; i < n; i += 8) {  
    __m256i x = _mm256_load_si256((__m256i*) &a[i]);  
    __m256i y = _mm256_load_si256((__m256i*) &b[i]);  
    __m256i z = _mm256_add_epi32(x, y);  
    _mm256_store_si256((__m256i*) &c[i], z);  
}
```

Если предположить, что в первом варианте начало массива совпадает с началом кэш-линии, а её размер 64 байта, то примерно половина `loadu` и `storeu` будут «плохими», связанными с загрузкой сразу двух кэш-линий. В итоге тестов невыровненная версия работает на 30 процентов медленней.

Чтобы продемонстрировать прирост производительности можно привести пример ускорения редукционной операции. Например необходимо сложить поэлементно массив целых чисел, это можно сделать следующим кодом:

```
#define ARR_SIZE 1600000000
#define CMP_VAL 1

int main(){
    int64_t cnt = 0;

    short *arr = malloc( ARR_SIZE * sizeof(short) );
    for (int i = 0; i < ARR_SIZE; ++i)
        arr[i] = i%2?10:1;

    double timeStart, timeEnd;
    timeStart = omp_get_wtime();

    for ( int i = 0; i < ARR_SIZE; ++i )
        if ( arr[i] == CMP_VAL )
            ++cnt;

    printf( "%ld\n", cnt);

    timeEnd = omp_get_wtime();
    printf( "Время выполнения %lf\n", timeEnd - timeStart);
}
```

Векторизированный вариант кода с выровненной памятью:

```
#define ARR_SIZE 1600000000
#define CMP_VAL 1

int main(){
    short *arr = aligned_alloc( 16, ARR_SIZE * sizeof(short) );
    for (int i = 0; i < ARR_SIZE; ++i)
        arr[i] = i%2?10:1;

    double timeStart, timeEnd;
    timeStart = omp_get_wtime();

    __m128i sseVal = _mm_set1_epi16( CMP_VAL );
    __m128i tmp = _mm_set1_epi16( 0 );
    int acc = 0;
    for ( int i = 0; i < ARR_SIZE; i+=8 ){
        __m128i sseArray = *(__m128i *) &arr[i];

        __m128i sseMask = _mm_cmpeq_epi16(sseVal, sseArray);
        __m128i add = _mm_and_si128( sseArray, sseMask );
        tmp = _mm_add_epi16( tmp, add );

        if( i % 32768 == 0 ){

            for( int j = 0 ; j < 8 ; ++j ){
                acc += *((short*)(&tmp) + j);
            }

            tmp = _mm_set1_epi16( 0 );
        }
    }
    for( int j = 0 ; j < 8 ; ++j ){
        acc += *((short*)(&tmp) + j);
    }
    tmp = _mm_set1_epi16( 0 );
    printf( "%d\n", acc);
    timeEnd = omp_get_wtime();
    printf("Время выполнения %lf\n", timeEnd - timeStart);
}
```


Благодаря тому, что память выровнена, возможно сделать прямой каст восьми элементом массива к `__m128i`. Далее необходимо реализовать ветвление, как это происходит в не векторизованном коде. Ветвление можно реализовать с помощью встроенной функции `_mm_cmpeq_epi16`, которая сравнит все восемь значений с ответными значениями, заранее заданными в `sseVal`. В случае соответствия все 16 бит данного числа будут взведены (`0xFFFF`), в противном случае значение будет 0 (`0x0`). Чтобы не прибавить лишнего значения, необходимо маскировать вектор получившейся маской с помощью `_mm_and_si128`. Далее вектор прибавляется к аккумулятору с помощью встроенной функции `_mm_add_epi16`. Маскирование не суммируемых чисел позволяет избежать переполнения 16 битных компонент вектора. В нужный момент векторизованный аккумулятор можно обнулить, предварительно сложив элементы в глобальный аккумулятор.

Задать вектор из разных элементов, не лежащих в соседней памяти, также возможно через функцию `_mm_set_epi`, она имеет модификации для 8, 16, 32, 64 чисел. Соответственно функции векторной сложения, вычитания, операции И, операции ИЛИ и тд реализованы для всех модификаций.